

Club Brésil

Bolão da Copa do Mundo 2026

Documentação de Arquitetura

Guia completo de tecnologia, hospedagem e funcionamento

Documento de referência e estudo para apresentar o projeto: o que foi construído, com quais tecnologias, onde está hospedado e por quê.

Enzo Vendramin

Junho de 2026

Sumário

1	Visão geral	2
2	Tecnologias e linguagens	2
3	Arquitetura geral	2
4	Modelo de dados	3
5	Regras de negócio	3
5.1	Pontuação	3
5.2	Bloqueio e edição de palpites	4
5.3	Visibilidade dos palpites	4
5.4	Ranking e empates	4
6	Estrutura de telas	4
7	Autenticação e segurança	4
8	Hospedagem e infraestrutura	5
9	Os dois bolões: França e Brasil	5
10	Fluxo de atualização (publicação)	6
11	Web Analytics	6
12	Custos	6
13	Glossário	6

1. Visão geral

O **Club Brésil** é uma plataforma web privada de bolão para a Copa do Mundo de 2026, pensada para grupos pequenos (cerca de 20 a 50 participantes). Substitui planilhas, formulários e mensagens soltas por um produto único, moderno e simples, no qual cada participante:

- cria uma conta e aguarda aprovação do administrador;
- registra palpites de placar para os jogos;
- acompanha sua pontuação, o ranking geral e o pódio.

Há um único **administrador** (você), que aprova participantes, cadastra jogos, lança resultados e gerencia o sistema. Existem hoje **dois bolões independentes** rodando com o mesmo código: um para amigos na França e outro para amigos no Brasil (detalhes na Seção 9).

Princípio de projeto

Toda decisão priorizou simplicidade, clareza e velocidade de uso no celular, mesmo para pessoas sem familiaridade com tecnologia. “Menos é mais” foi a régua de cada escolha.

2. Tecnologias e linguagens

O projeto usa uma stack moderna e enxuta, na qual **um único programa** cuida tanto da tela (*frontend*) quanto da lógica de servidor (*backend*). Isso reduz custo, manutenção e complexidade.

Camada	Tecnologia e por quê
Linguagem	TypeScript — JavaScript com tipos. Dá segurança às regras de pontuação e aos formulários, evitando erros bobos.
Framework	Next.js 15 (App Router) sobre React 19 . Um só projeto para frontend e backend, com renderização no servidor (rápido no celular) e <i>Server Actions</i> (formulários enviam direto para funções de servidor, sem precisar de uma API separada).
Estilo	Tailwind CSS 4 . Design consistente e responsivo sem bibliotecas pesadas de componentes.
Banco de dados	PostgreSQL (gerenciado pela Neon) acessado via Prisma ORM . O Prisma traduz o banco em código com tipos, e permitiu começar em SQLite no desenvolvimento e migrar para PostgreSQL na produção mudando praticamente uma linha.
Autenticação	Sistema próprio com JWT em cookie (biblioteca <i>jose</i>) e senhas protegidas por bcrypt . Sem serviços externos, sem custo, sem a complexidade do “login com Google”.
Bandeiras	Imagens do serviço gratuito <i>flagcdn.com</i> . Aparecem iguais em qualquer aparelho (emojis de bandeira não funcionam no Windows).

Por que “tudo em um” (Next.js)?

Em vez de manter um site e um servidor de API separados, o Next.js une os dois. Resultado: um único artefato para publicar, menos peças que podem quebrar e um aplicativo muito leve (aproximadamente 100 kB de JavaScript no navegador), o que o torna rápido até em conexões ruins de celular.

3. Arquitetura geral

O caminho de uma ação do usuário, do toque na tela até o banco de dados:

Celular/navegador → **Vercel** (executa o Next.js na região mais próxima) → **Server Action** (valida permissões e regras) → **Prisma** → **PostgreSQL/Neon**

Pontos-chave da arquitetura:

- **Validação sempre no servidor.** Nenhuma regra depende do navegador: aprovação de usuário, papel de administrador, bloqueio de palpites e pontuação são verificados no servidor a cada ação. O cliente nunca é “confiável”.
- **Status do jogo é calculado, nunca armazenado.** “Aberto”, “Fechado” e “Finalizado” derivam do horário atual e do placar. Assim, o bloqueio de palpites acontece **exatamente** no início da partida, sem precisar de nenhuma tarefa agendada.
- **Pontuação materializada.** Quando o administrador lança um resultado, o sistema calcula e grava os pontos de cada palpite e recalcula o ranking. As telas de leitura ficam, então, triviais e rápidas.

4. Modelo de dados

Cinco entidades organizam tudo, sempre separadas por edição (Copa 2026, e futuras 2030, 2034...):

Entidade	O que guarda
User	Nome (exibido no ranking), nome de usuário (login), senha protegida, papel (administrador ou participante) e situação (pendente ou aprovado).
Edition	Uma edição do bolão (ex.: Copa 2026), a seleção campeã e a configuração de trava do “chute do campeão”.
Match	Um jogo: as duas seleções, data/hora (em UTC), fase, placar oficial e o controle de liberação de palpites.
Prediction	O palpite de um usuário em um jogo (gols A e B) e os pontos já calculados. Único por usuário e jogo.
Participation	Liga um usuário a uma edição e guarda seus totais: pontos, placares exatos, acertos, posição atual, posição anterior (para a evolução) e o chute do campeão.

5. Regras de negócio

5.1 Pontuação

A pontuação é **configurável por bolão** (ver Seção 9). A regra geral:

- **Placar exato** (acertou os gols dos dois lados): pontuação cheia;
- **Acertou o resultado** (quem venceu, ou empate, sem o placar exato): pontuação menor;
- **Errou:** zero;
- **Chute do campeão** (acertar quem levanta a taça): bônus somado ao ranking quando o administrador define a campeã.

As duas primeiras regras **não somam**: quem acerta o placar exato recebe a pontuação cheia, que já inclui ter acertado o vencedor.

Mata-mata e pênaltis

Conta apenas o placar do tempo regulamentar mais prorrogação. Disputa por pênaltis não altera o placar para fins de pontuação: um 1 × 1 decidido nos pênaltis continua sendo empate.

5.2 Bloqueio e edição de palpites

Cada jogo abre para palpites automaticamente alguns dias antes (evitando uma tela enorme de uma vez) e **trava no horário exato do apito inicial**. Antes disso, o participante pode editar livremente, a qualquer momento e aos poucos. O administrador também pode liberar ou bloquear manualmente cada jogo.

5.3 Visibilidade dos palpites

Antes do início da partida, ninguém vê o palpite dos outros. A partir do apito inicial, os palpites de todos ficam visíveis (na aba Histórico, em um clique expansível) — e assim permanecem após o fim do jogo, aumentando a transparência.

5.4 Ranking e empates

O ranking é recalculado automaticamente a cada resultado lançado. Em caso de empate de pontos, ambos ocupam a **mesma posição** (ex.: 1º, 1º, 3º), sem critérios de desempate. O sistema guarda a posição anterior para mostrar a evolução (↑, ↓ ou estável) a cada rodada.

6. Estrutura de telas

Rota	Função
/	Página pública (sem login): pódio, ranking e resultados.
/entrar, /cadastro	Login e criação de conta.
/aguardando	Tela de espera até a aprovação do administrador.
/inicio	Próximo jogo, palpites pendentes, pódio e ranking resumido.
/palpites	Jogos abertos, agrupados por dia; “Salvar todos os palpites”.
/ranking	Pódio e tabela completa com evolução.
/historico	Jogos iniciados, seu palpite, pontos e palpites de todos.
/agenda	Todos os jogos da Copa, dia a dia (inclusive os que não valem palpite), como consulta.
/senha	Trocar a própria senha.
/admin/*	Painel: participantes, jogos, resultados e visão de palpites.

A navegação principal é uma barra inferior fixa (padrão de aplicativo nativo), com campos e botões grandes, teclado numérico automático nos placares e respeito à área segura do iPhone. Tudo **mobile-first**.

7. Autenticação e segurança

- **Login** por nome de usuário e senha; senhas guardadas como *hash* bcrypt (nunca em texto puro).
- **Sessão** em cookie `httpOnly` assinado (JWT), válido por 30 dias — não acessível por scripts no navegador.
- **Aprovação manual**: novos cadastros ficam pendentes até o administrador aprovar.
- **Limite de tentativas** de login (proteção contra adivinhação de senha) e **cabeçalhos de segurança** (anti-*clickjacking*, etc.).
- **HTTPS** automático em produção.
- A página pública não expõe dados privados — apenas nome e pontuação.

8. Hospedagem e infraestrutura

Três serviços, todos no **plano gratuito**:

Serviço	Papel	Por que esse
GitHub	Guarda o código-fonte (repositório privado).	Padrão da indústria; integra-se à Vercel para publicar automaticamente.
Vercel	Hospeda e executa o aplicativo.	É a criadora do Next.js (encaixe perfeito), com HTTPS e rede global automáticos. Publica sozinha a cada atualização do código.
Neon	Banco de dados PostgreSQL gerenciado.	PostgreSQL “sem servidor”, com plano gratuito generoso e backups; funciona bem com a Vercel.

Por que não foi usado SQLite em produção?

No desenvolvimento, o banco era um simples arquivo (SQLite) — ótimo para testar localmente. Mas a Vercel executa o código em ambiente “sem servidor”, onde não há um disco fixo para guardar esse arquivo. Por isso a produção usa PostgreSQL (Neon), um banco de verdade acessível pela rede. A troca custou praticamente uma linha de configuração, graças ao Prisma.

A lição da latência

No primeiro deploy, o código rodava nos EUA e o banco na Europa: cada tela cruzava o Atlântico várias vezes e demorava $\approx 2,2$ s. Ao fixar a execução na **mesma região** do banco e dos usuários, o tempo caiu para $\approx 0,15$ s — cerca de $10\times$ mais rápido. Daí a importância de código e banco ficarem geograficamente juntos.

9. Os dois bolões: França e Brasil

O **mesmo código** alimenta dois bolões totalmente independentes (dados, participantes e rankings separados). As diferenças vêm apenas de configuração, não de código duplicado:

	Bolão França	Bolão Brasil
Servidor (Vercel)	Frankfurt (fra1)	São Paulo (gru1)
Banco (Neon)	Frankfurt (UE)	São Paulo (BR)
Fuso exibido	Horário de Paris	Horário de Brasília
Placar exato	3 pontos	5 pontos
Acertar resultado	1 ponto	2 pontos
Chute do campeão	10 pontos	10 pontos
Web Analytics	Ativado	Desativado

Tecnicamente, cada bolão é um *projeto* separado na Vercel, ligado a uma *branch* do mesmo repositório (**main** para a França, **brasil** para o Brasil) e a um banco Neon próprio. Comportamentos como fuso horário e pontuação são lidos de **variáveis de ambiente** (APP_TZ, SCORE_EXACT, SCORE_OUTCOME, NEXT_PUBLIC_ANALYTICS) — por isso o mesmo código se comporta diferente em cada lugar, sem precisar mantê-los à parte.

Um não atrapalha o outro

Por serem projetos e bancos separados, os dois bolões não compartilham servidor nem dados. Carga, lentidão ou qualquer problema em um **não afeta** o outro.

10. Fluxo de atualização (publicação)

O desenvolvimento e a publicação seguem um ciclo simples e seguro:

1. A alteração é feita e testada **localmente** (no computador), sem afetar os sites no ar.
2. Ao enviar o código para o GitHub (`git push`), a Vercel **detecta e publica** a nova versão em ≈ 2 minutos, **sem derrubar o site e sem tocar nos dados** (que vivem no banco, separados do código).
3. Mudanças arriscadas podem ser testadas antes em uma **URL de prévia** (cópia isolada que a Vercel gera automaticamente).
4. Se algo der errado, a Vercel permite **reverter** para a versão anterior em segundos.

Melhorias comuns são aplicadas à França e replicadas ao Brasil com a junção das *branches*; cada site se atualiza na sua região.

11. Web Analytics

O bolão da França usa o **Web Analytics** da Vercel (gratuito): mostra número de visitantes, páginas mais acessadas, país e tipo de aparelho. É **amigável à privacidade** — não usa cookies nem identifica indivíduos, adequado às regras de privacidade da União Europeia. É ativado apenas onde a variável `NEXT_PUBLIC_ANALYTICS` está definida.

12. Custos

Item	Custo
GitHub (repositório privado)	R\$ 0
Vercel (2 projetos, plano Hobby)	R\$ 0
Neon (2 bancos, plano Free)	R\$ 0
Total mensal	R\$ 0

Os planos gratuitos comportam, com folga, dois grupos de 20 a 50 pessoas. O modelo só precisaria de plano pago se o número de usuários crescesse muito (centenas de milhares de acessos), o que não é o caso.

13. Glossário

Frontend

A parte visível: as telas com que o usuário interage.

Backend

A parte invisível: a lógica de servidor e o acesso ao banco.

Framework

Conjunto de ferramentas que estrutura o desenvolvimento (aqui, o Next.js).

ORM

Tradutor entre o banco de dados e o código (aqui, o Prisma).

Server Action

Função de servidor chamada diretamente por um formulário, sem uma API separada.

Deploy

Publicar uma nova versão do aplicativo no ar.

Branch

Uma “linha” de código no Git; usamos uma para cada bolão.

Variável de ambiente

Configuração definida fora do código (ex.: fuso, pontuação), que muda o comportamento sem alterar o programa.

Serverless

Modelo em que o provedor executa o código sob demanda, sem um servidor fixo a administrar.

JWT

“Crachá” digital assinado que comprova a sessão do usuário.

Hash (bcrypt)

Transformação irreversível da senha, para nunca guardá-la em texto puro.